

EECS 6412 – Data Mining

Assignment 2 – Methodology Report

Omkumar M. Patel (222110936), Michael Murphy (222636120)

1 Preprocessing

The risk dataset contains a mix of categorical attributes, timestamp features, financial values, previous customer behaviour signals, and several columns that require domain-aware cleaning. The raw data also included missing values encoded as “?”, non-standard date formats, payment-specific fields, and categorical variables disguised as numerical. Because of these issues, we extended the base preprocessing so that all important fields are transformed into stable, interpretable numerical features before passing them to the generic preprocessing function.

1.1 Initial Cleaning and ID Handling

We removed ORDER_ID from the feature matrix (kept separately for producing the final submission) and replaced "?" uniformly with missing values so downstream steps treat missingness consistently. This step ensures that the model does not mistakenly learn from identifiers or hidden encodings.

1.2 Handling Multiple Missing Values in Various Columns

Initially our code checks each column’s missing-value (“?”) ratio compared to total values to indicate lost information. However, after evaluating we didn’t drop all the columns from drop_col_list who had more than 20% missing values. Instead, we went on to find the correlations among them and found useful insights.

We computer correlation matrix and saw high correlation among Group 1 and 2. Wherein the features of Group1 are DATA_LOADER, MAHN_AKT, MAHN_HOECHST, reflecting relation among new customers and reminders sent to them. And Group2: Z_CARD_ART and Z_LAST_NAME represent payment mode, which might be a strong signal for fraud detection.

We did dropp ANUMMER_02 to 10 columns as they had more than 70% missing values, with no significant relationships with other features and similar data available in AMOUNT_ORDER.

Two fields required explicit transformation:

TIME_ORDER: Converted from HH:MM strings into hour and minute numerical features. Missing or unparseable times were filled using the most common valid time.

DATE_LORDER: Converted into year, month, and day components. A missingness indicator was added because “no previous order” is meaningful information.

These decompositions allow the model to learn ordering patterns (e.g., certain hours correlate with different risk levels) while avoiding issues with raw string formats.

1.3 Customer Age Calculation

Instead of using B_BIRTHDATE directly, we computed the customer’s age in days relative to their most recent order date. This transformation provides a stable numerical feature and is generally more informative than embedding the raw date. An additional indicator captures missing birthdates.

1.4 Categorical Corrections for Payment and Behaviour Features

Several attributes depend on the payment method:

- Card-specific fields were set to a neutral category when the payment method was not a credit card.
- Cheque-specific verification fields were treated similarly.
- Card-validity information was broken into month and year values.

These changes prevent the model from drawing incorrect conclusions from missing or irrelevant entries and allow consistent encoding across all samples.

1.5 Correct Treatment of Product Identifiers

ANUMMER_01 appears numeric but represents product IDs. Treating it as a continuous feature would distort the meaning (e.g., product “200126” is not “twice” product “100064”). We explicitly converted it to a categorical variable to ensure proper one-hot encoding.

1.6 Reminder-Count Features

Dunning reminder counts (MAHN_AKT, MAHN_HOECHST) were interpreted as zero when missing, since missing is most naturally read as “no reminder sent yet.”

1.7 Numeric Conversions, Feature selection and Final General Preprocessing

After these manual corrections, the dataset was passed through the established preprocessing function (from Q2), performing missing-value imputation, normalization of numeric attributes, and one-hot encoding of categorical columns. And with the selected features, we generate the final feature matrix having 607 dimensions for the training data.

This combined approach preserves the intent of the original routines while ensuring that each complex column is meaningfully represented.

2 Learning Techniques and Evaluation Framework

2.1 Class Imbalance and Cost Structure

The training set is highly imbalanced (~5.8% “yes” class), and as per the assignment, we needed to compute a cost matrix in which misclassifying a “yes” case costs ten times more than falsely flagging a “no” case. Because of this, all evaluation and model comparison must be done through a cost-based objective, not pure accuracy. Used stratified 10-fold cross-validation ensured that label imbalance was preserved across folds.

We encoded this imbalance in two ways:

1. **Class weights** during model training (greater penalty on the positive class).
2. **A custom scoring function** computing average misclassification cost under the provided cost matrix.

2.2 Classifiers Evaluated

We trained and evaluated the following classifiers:

- Decision Tree
- Logistic Regression
- Random Forest

These were chosen because we worked with them in the earlier questions, and they offer complementary strengths like interpretability, and linear separability supporting a realistic comparison. Before threshold tuning, Logistic Regression achieved the lowest cost, but this relied on the default probability threshold of 0.5, which is not necessarily optimal for asymmetric cost problems.

2.3 Threshold Fine-Tuning

Default thresholds assume equal cost for both classes. Our cost matrix implies that the optimal threshold should be closer to ~ 0.09 (derived from the Bayesian decision rule). Rather than hard-coding this value, we used out-of-fold predicted probabilities to sweep through 100 possible thresholds and compute their average misclassification cost. This allowed us to identify the cost-minimizing threshold for each model.

The best thresholds were approximately:

- Decision Tree: 0.01
- Logistic Regression: 0.52
- Random Forest: 0.08

Corresponding cross-validated approximate average costs:

- Decision Tree: 2.785
- Logistic Regression: 2.347
- Random Forest: 2.317

After threshold optimization, Random Forest was leading, and Logistic Regression came quite closer to it.

3. Additional Validation

To ensure our threshold choices were not simply fitting the cross-validation folds, we (used AI) performed an independent train/validation split and evaluated each tuned model on the untouched validation set.

Key observations:

- With the fine-tuned threshold of 0.52, Logistic Regression achieved the **lowest validation average cost per sample (2.3483)**, slightly outperforming Random Forest (**2.3975**). This indicates that LR generalizes very well under the given asymmetric cost matrix and predicts "yes" for about 18% of samples.
- Decision Tree predicted "yes" at exactly the baseline rate but suffered high false negatives, again increasing cost.
- Random Forest predicted "yes" for about 23% of samples, much higher than the true rate but far more controlled than the Decision tree and matched its cross-validated performance.

Thus, **LR comes out on top during validation**, slightly outperforming RF by cost, while DT remains weakest. This shows LR's tuned threshold corrects its earlier aggressive behaviour and yields the most cost-efficient decision boundary on held-out data.

4. Final Model Selection

We selected Logistic Regression with a tuned threshold of approximately 0.52 as our final classifier for the risk-prediction task. This choice of model was due to it having:

- The **lowest observed average misclassification cost of 2.3483**.
- Higher precision than Random Forest and quite balanced recall levels appropriate for catching high-risk cases,
- A reasonable positive prediction rate aligned with the nature of cost-sensitive decision-making.
- Aligns quite well with the objective of minimizing overall misclassification cost.

5. Conclusion

Our improved pipeline with extended feature engineering and cost-sensitive optimization helped us produce quite a stable dataset compared to the original for our selected models and supported comparison of classifiers as per the provided misclassification cost matrix.

Logistic Regression emerged as the most cost-effective model, providing competitive recall while keeping the false-positive rate under control. The tuned threshold allows the model to reflect the actual business cost structure, resulting in a more practical and accurate risk classifier for the given dataset. This final model, together with the preprocessing and feature-selection choices described above, is a well-justified risk classifier for the given dataset.